Docker Workshop

From Zero to Hero: A Comprehensive Guide to Docker

August 8, 2025

Workshop Agenda

1. Introduction to Docker
2. Installing Docker
3. Your First Container
4. Images & Dockerfiles
5. Data Persistence
6. Networking & Ports
7. Compose Essentials
8. Registries & Distribution
9. Lifecycle & Optimization
10. Security & Conclusion

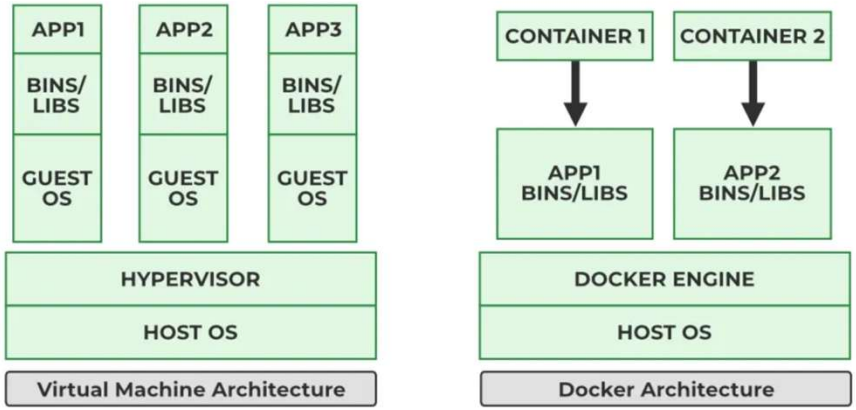
Introduction to Docker

What is Containerization?

- Abstracts application environment for consistent deployment
- Isolates processes and dependencies while sharing host OS kernel
- Lightweight, fast to start, and resource efficient

Key Use Cases

- Microservices architecture
- CI/CD pipelines
- Portable development environments



Installing Docker

Windows / macOS

- Download Docker Desktop from docker.com
- Follow the installer wizard
- Enable WSL 2 backend on Windows when prompted

Linux

- Use your distribution's package manager:

```
# Ubuntu/Debian
sudo apt install docker-ce docker-compose
```

- Add your user to the docker group:

```
sudo usermod -aG docker $USER
```

Verification

- Run "docker --version" to verify the client
- Try "docker run hello-world" to test the engine

Your First Container

Pull

```
docker pull hello-world
```

Run

```
docker run hello-world
```

Explore

```
docker ps -a
```

Clean

```
docker rm <id>
```

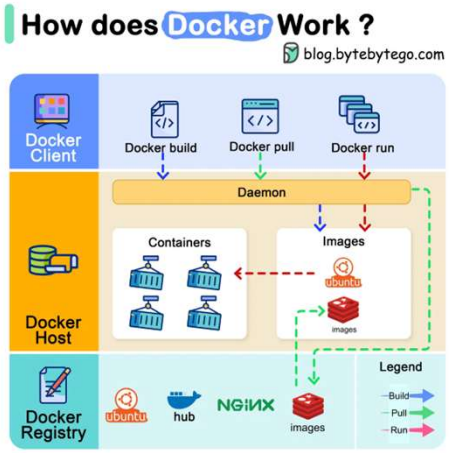
Interactive vs Detached

```
# Interactive mode
docker run -it ubuntu bash

# Detached mode
docker run -d nginx
```

Cleaning Up

```
# Remove stopped containers
docker container prune

# Remove unused resources
docker system prune
```

Images & Dockerfiles



Images & Layers

- Read-only templates with multiple stacked layers
- Union filesystem presents layers as single view
- Tag images with repository:tag (e.g., nginx:1.25)

Dockerfile Instructions

- FROM: Sets the base image
- RUN: Executes shell commands
- COPY/ADD: Brings files into the image
- CMD/ENTRYPOINT: Declares default process

Multi-stage Builds

```
FROM golang:1.21-alpine AS builder
WORKDIR /src
COPY . .
RUN go build -o app ./...

FROM alpine:latest
WORKDIR /app
COPY --from=builder /src/app .
EXPOSE 8080
CMD ["./app"]
```

Data Persistence

Volume Types

Named Volumes

Created and managed by Docker, stored in /var/lib/docker/volumes

```
docker volume create mydata
```

Bind Mounts

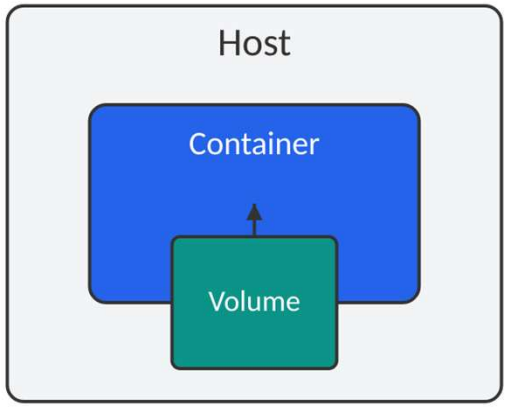
Map a specific host directory into the container

```
docker run -v /host/path:/container/path
```

Anonymous Volumes

Created automatically when mounting a path without a name

```
docker run -v /container/path
```



Volumes persist data outside the container lifecycle

Networking & Ports

Network Types

Bridge

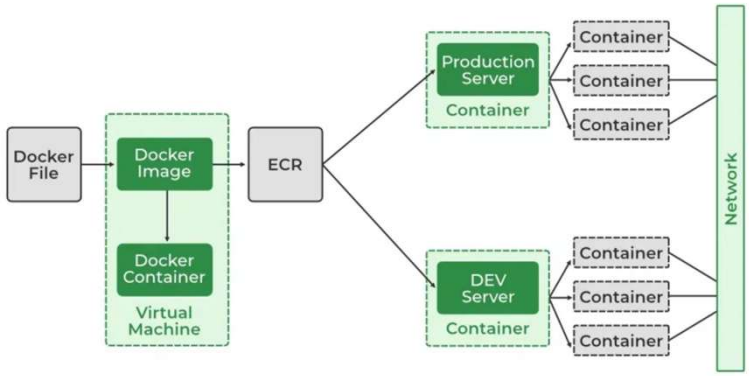
Default network with private IPs for containers.

Host

Uses host's network stack directly.

Overlay

Spans multiple Docker hosts using VXLAN.



Port Mapping

```
# Publish ports
docker run -p 8080:80 nginx

# Document ports in Dockerfile
EXPOSE 80
```

Custom Networks

```
# Create network
```

Compose Essentials

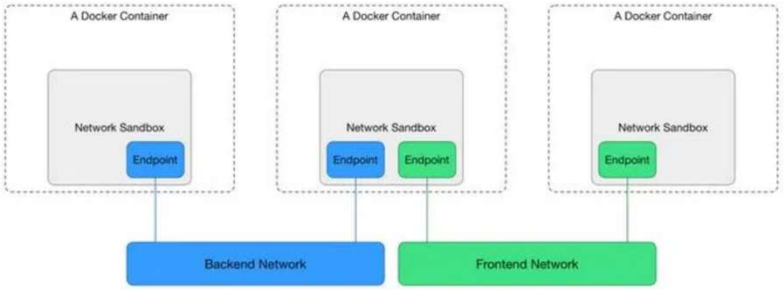
Why Compose?

- Orchestrates multi-container applications
- Eliminates lengthy command lines
- Ensures services start in correct order

Key Commands

- docker compose up -d
- docker compose ps
- docker compose logs
- docker compose down

Three-tier Example




```
version: "3.9"
services:
  redis:
    image: redis:alpine
    volumes:
      - redisdata:/data
  api:
    build: ./api
    ports:
      - "3000:3000"
    depends_on:
      - redis
  web:
    build: ./web
    ports:
      - "8080:80"
    depends_on:
      - api
volumes:
  redisdata:
```

Registries & Distribution

Registry Types

Docker Hub

Default public registry. Free for open-source projects.

Private Registries

Self-hosted (Harbor, Nexus) or cloud services (AWS ECR, GCR, ACR).

Push/Pull Workflow

1. Authenticate with registry

```
docker login <registry>
```

2. Tag your image for the registry

```
docker tag myapp:1.0 registry.example.com/myapp:1.0
```

3. Push image to registry

```
docker push registry.example.com/myapp:1.0
```

4. Pull image from registry

```
docker pull registry.example.com/myapp:1.0
```

Lifecycle & Optimization

Container States

Start & Restart

```
docker start <id>
```

```
docker restart <id>
```

Attach & Exec

```
docker attach <id>
```

```
docker exec -it <id> bash
```

Removal & Cleanup

```
docker rm <id>
```

```
docker system prune
```

Optimization Techniques

- Choose minimal base images (alpine, distroless)
- Combine related commands in single RUN instruction
- Clean up package caches in the same layer
- Use multi-stage builds to separate build from runtime
- Order instructions from least to most frequently changing
- Use .dockerignore to exclude unnecessary files
- Tag images explicitly instead of using latest

Security & Conclusion

Security Best Practices

- Run as non-root
- No secrets in images
- Scan for vulnerabilities
- Read-only filesystems

Next Steps

- Explore Kubernetes orchestration
- Automate with CI/CD pipelines

Questions?

Thank you for attending!

Key Takeaways

Portability

Docker packages applications with dependencies for consistent execution across environments.

Efficiency

Containers share the host OS kernel, making them lightweight and resource-efficient.

Scalability

Docker Compose enables easy scaling of containerized applications.

DevOps Integration

Containers bridge development and operations with consistent environments.